



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Laboratorio S05

Mexi Banco con Compose

MATERIA

Sistemas Distribuidos

Alumno

Reyes Vázquez Fernando

PROFESOR

Oscar Manuel Ruiz Hurtado

FECHA DE ENTREGA | 27 DE SEPTIEMBRE DE 2026

GRUPO 02 | SEMESTRE 2027-1

Preparación del proyecto

```
fernandoreyesvazquez@MacBookAir ~ % cd ~
fernandoreyesvazquez@MacBookAir ~ % git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 646.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
warning: refs/tags/v05.1 cd46b8b91a907870f8262c3b8acfb17caf2405cda is not a commit!
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

fernandoreyesvazquez@MacBookAir ~ % cd mexi-banco
fernandoreyesvazquez@MacBookAir mexi-banco % pwd
/Users/fernandoreyesvazquez/mexi-banco
fernandoreyesvazquez@MacBookAir mexi-banco % ls
docker-compose.yml  Dockerfile          k8s                 mvnw                mvnw.cmd            pom.xml             README.md           src
```

Figura 1. Clonación del repositorio Mexi Banco v05.1 y verificación de los archivos del proyecto.

Se clonó la versión v05.1 del repositorio de Mexi Banco y se verificó la estructura inicial del proyecto.

Despliegue con Docker Compose

```
[+] Building 89.4s (19/19) FINISHED
=> [internal] load local bake definitions. 0.0s
=> => reading from stdin 522B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 440B 0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 4.4s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine 4.6s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
[> [builder 1/8] FROM docker.io/library/eclipse-temurin:21-jdk-alpine#sha256:0bfcc69a758aa8710e5c474032d28400a8bb08874766f9e8b1642ac2cf2093159 22.5s
=> resolve docker.io/library/eclipse-temurin:21-jdk-alpine#sha256:0bfcc69a758aa8710e5c474032d28400a8bb08874766f9e8b1642ac2cf2093159 0.0s
=> sha256:794db8f647ba05f395f78c45b9f72beed4621ec584a8dfcb22f4f7b2c2558 / 2.46KB 0.2s
=> sha256:1656fa543863ab3ca42a678399a37fa7fb7933ea9v708bcf3f32a3ebccdcb56 / 129B / 129B 0.2s
=> sha256:8438a5fa39bf9fc48be02accb3ffad4decd6fd7a099f7ae68331df6acdcdc68 / 156.34MB / 156.34MB 28.3s
=> sha256:c807d068ee33e5d597c7313f9cfc87af219dcf1e31b59a3bcded411088d43e / 21.42MB / 21.42MB 3.6s
=> extracting sha256:c807d068ee33e5d597c7313f9cfc87af219dcf1e31b59a3bcded411088d43e 0.3s
=> extracting sha256:8438a5fa39bf9fc48be02accb3f7fad4decd6fd7a099f7ae68331df6acdcdc68 1.1s
=> extracting sha256:1656fa543863ab3ca42a678399a37fa7fb7933ea9v708bcf3f32a3ebccdcb56 0.0s
=> extracting sha256:794db8f647ba05f395f78c45b9f72beed4621ec584a8dfcb22f4f7b2c2558 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 52.16kB 0.0s
[> [stage-1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine#sha256:51ab5e3302e7141ce646a3ea850e8b5cd48eafbc3c0c90dd79e637684e4555 7.7s
=> resolve docker.io/library/eclipse-temurin:21-jre-alpine#sha256:51ab5e3302e7141ce646a3ea850e8b5cd48eafbc3c0c90dd79e637684e4555 0.0s
=> sha256:fc16ccc3e3441becdb6c8779a883034adebb6d664f4f5b3c69594e48 / 127B / 127B 0.1s
=> sha256:23e297f795e64f0a334800d4ba28b08a63754e4eedeca85080d7e88357a7099e / 2.46KB / 2.46KB 0.1s
=> sha256:809353e827a60532ea55b749dc0cf05d297b1821ef8fa0e3e64be39e8ec09d / 52.32MB / 52.32MB 7.1s
=> sha256:c5863823da68bce2695869ca260e05d9ef1927f83f9f0b7fbaafce366d8c / 16.99MB / 16.99MB 3.1s
=> sha256:f37d7dee7862a3a67a8a3787e7c2ea7080d7bd4f4fba393c0f000 / 4.19MB / 4.19MB 1.1s
=> extracting sha256:a9986ac6d737fbbada7862a64be71683427c7c2ea7080d7bd4f4fba393c0f000 0.1s
=> extracting sha256:c5063e83da68bce2695869ca260e05d9ef1927f83f9f0b7fbaafce366d8c 0.2s
=> extracting sha256:879353e827a60532ea55b749dc0cf05d297b1821ef8fa0e3e64be39e8ec09d 0.0s
=> extracting sha256:fc16ccc3e3441becdb6c8779a883034adebb6d664f4f5b3c69594e48 0.0s
=> extracting sha256:23e297f795e64f0a334800d4ba28b08a63754e4eedeca85080d7e88357a7099e 0.0s
[> [stage-2/3] WORKDIR /app 0.0s
[> [builder 2/8] WORKDIR /app 0.3s
[> [builder 3/8] COPY pom.xml . 0.0s
[> [builder 4/8] COPY mvnw . 0.0s
[> [builder 5/8] COPY mvnw.mvn . 0.0s
[> [builder 6/8] RUN ./mvnw dependency:go-offline -B 56.1s
[> [builder 7/8] COPY src ./src 0.1s
[> [builder 8/8] RUN ./mvnw clean package --skipTests 4.2s
[> [step-1/3/3] COPY --from=builder app/target/*-j.jar app.jar 0.1s
=> exporting to image 1.2s
=> => exporting layers 0.9s
=> => exporting manifest sha256:22ab264ffd369a212d9ab99f9dda01c9f146afla6d4a6badcce8879b2ec25d1a 0.0s
=> => exporting config sha256:eae9f8fce230498a0bedca3e2578c497806b0ba79baf22802c44a828df0790 0.0s
=> => exporting attestations sha256:af282264433fe54a32509a7089562766f30b383034f36139cf02a0b6724icf33 0.0s
=> => exporting manifest list sha256:b87161337af3632c707c1ffa4c8aac75c0729489941b07b09a28db643daf9cb 0.0s
=> => naming to docker.io/library/mexi-banco-app:latest 0.0s
=> => unpacking to docker.io/library/mexi-banco-app:latest 0.2s
[> resolving provenance for metadata file 0.0s
[+] Up 5/5
✔ Image mexi-banco-app Built 91.1s
✔ Volume mexi-banco_mexibanco_datos Created 0.0s
✔ Network mexibanco Created 0.0s
✔ Container mexi-banco-db-1 Healthy 5.0s
✔ Container mexi-banco-app-1 Started 0.0s
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
mexi-banco-app-1	mexi-banco-app	"java -jar app.jar"	app	39 seconds ago	Up 33 seconds (healthy)	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1	postgres:12-alpine	"docker-entrypoint.sh..."	db	39 seconds ago	Up 33 seconds (healthy)	5432/tcp

```
fernandoreyesvazquez@MacBookAir mexi-banco % docker compose ps
```

Figura 2. Servicios de Mexi Banco levantados con Docker Compose y verificados en estado healthy.

El sistema se levantó mediante Docker Compose. Se verificó que los servicios app y db alcanzaran el estado healthy, confirmando que la aplicación y PostgreSQL estaban funcionando correctamente.

Pruebas funcionales de la API

Arriba y respondiendo

Primero se comprobó que la aplicación estuviera disponible mediante el endpoint de salud, obteniendo 200 OK y estado UP. Después se creó una cuenta con saldo inicial de 1000 y se consultó nuevamente para comprobar que quedó registrada correctamente.



Figura 3. Verificación del estado de la aplicación mediante el endpoint de salud.

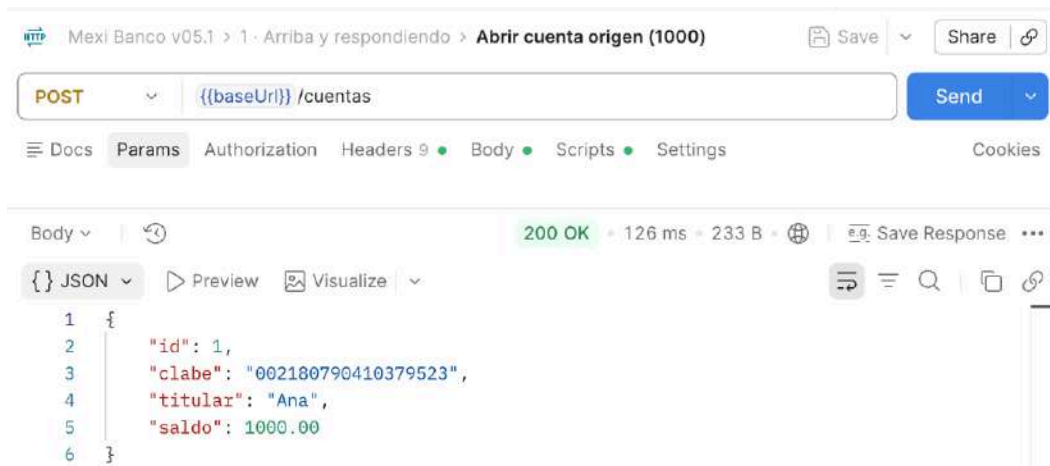


Figura 4. Creación de la cuenta origen con un saldo inicial de 1000.



Figura 5. Consulta de la cuenta origen después de su creación.

Transferencia interna

Se creó una segunda cuenta con saldo inicial de 500 y se realizó una transferencia de 200 desde la cuenta origen. Después de la operación, el saldo de la cuenta origen pasó de 1000 a 800, mientras que la cuenta destino pasó de 500 a 700. Finalmente, se consultó la transferencia para comprobar que quedó registrada correctamente.



Figura 6. Creación de la cuenta destino con saldo inicial de 500.

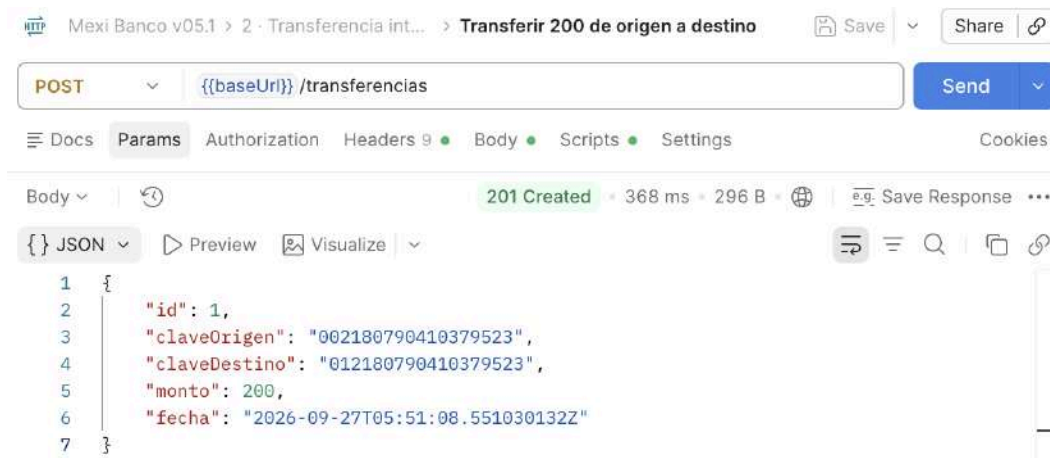


Figura 7. Transferencia de 200 de la cuenta origen a la cuenta destino.



Figura 8. Consulta de la cuenta origen después de la transferencia.



Figura 9. Consulta de la cuenta destino después de la transferencia.



Figura 10. Consulta de la transferencia realizada.

Idempotencia

Se envió un SPEI de 50 y posteriormente se repitió la misma solicitud utilizando la misma Idempotency-Key. Ambas peticiones devolvieron el mismo identificador, por lo que no se creó una segunda operación. El saldo final fue de 750, comprobando que el monto solo se descontó una vez.

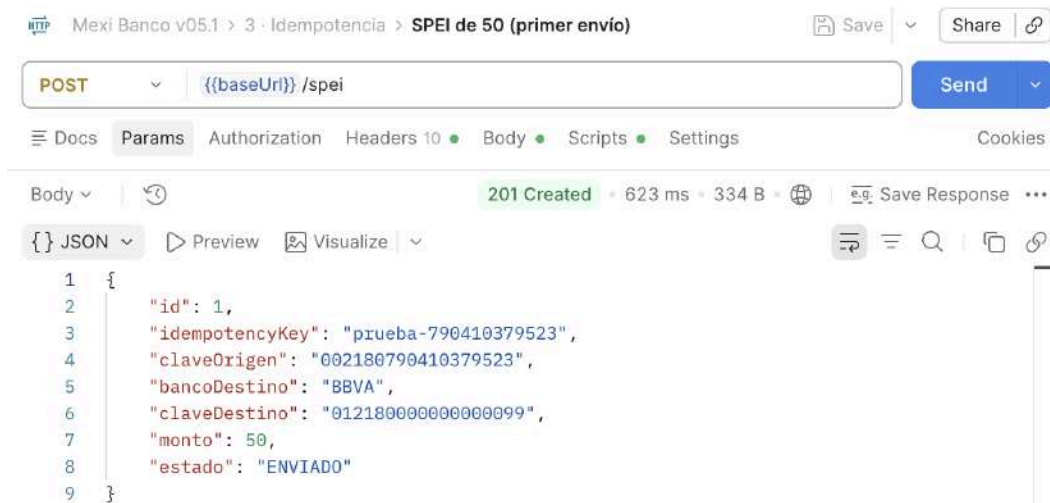


Figura 11. Primer envío de un SPEI de 50 utilizando una clave de idempotencia.



Figura 12. Reintento del mismo SPEI utilizando la misma clave de idempotencia.

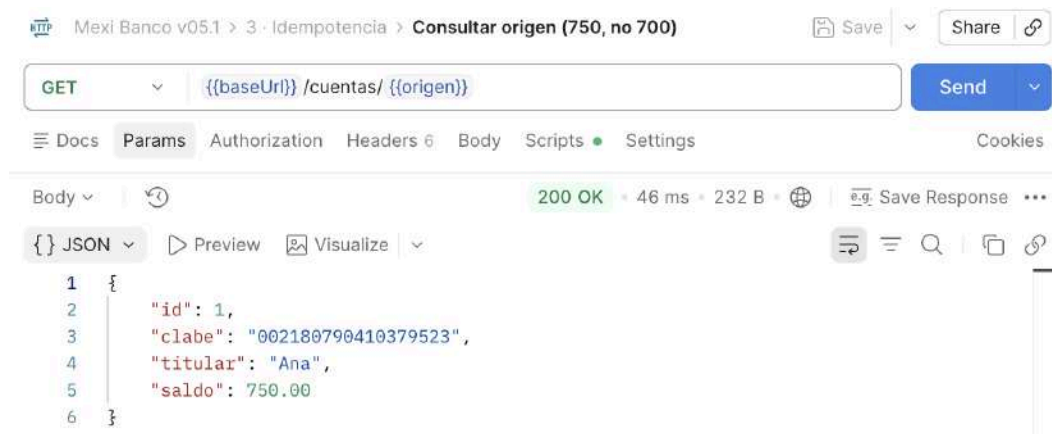


Figura 13. Consulta del saldo de la cuenta origen después del reintento del SPEI.

El monolito por dentro

En esta parte se revisó la información que dejaron las operaciones realizadas anteriormente dentro del sistema. Al consultar los movimientos de la cuenta origen se comprobó que únicamente se registró un SPEI y que los movimientos corresponden con el saldo actual de 750. También se consultaron las notificaciones, obteniendo dos avisos: uno por la transferencia interna y otro por el SPEI.

Mexi Banco v05.1 > 4 - El monolito por dentro > Estado de cuenta del origen

GET {{baseUrl}} /movimientos?clabe={{origen}}

200 OK • 124 ms • 712 B

JSON

```

1 [
2   {
3     "id": 5,
4     "clabeCuenta": "002180790410379523",
5     "tipo": "SPEI_ENVIADO",
6     "monto": -50.00,
7     "saldoResultante": 750.00,
8     "fecha": "2026-09-27T06:15:46.418675Z",
9     "referencia": "BBVA • 012180000000000099"
10  },
11  {
12    "id": 3,
13    "clabeCuenta": "002180790410379523",
14    "tipo": "TRANSFERENCIA_ENVIADA",
15    "monto": -200.00,
16    "saldoResultante": 800.00,
17    "fecha": "2026-09-27T05:51:08.548307Z",
18    "referencia": "a 012180790410379523"
19  },
20  {
21    "id": 1,
22    "clabeCuenta": "002180790410379523",
23    "tipo": "DEPOSITO",
24    "monto": 1000.00,
25    "saldoResultante": 1000.00,
26    "fecha": "2026-09-26T08:13:00.112920Z",
27    "referencia": "saldo inicial"
28  }
29 ]

```

Figura 14. Consulta de los movimientos registrados en la cuenta origen.

Mexi Banco v05.1 > 4 - El monolito por dentro > Avisos del origen

GET {{baseUrl}} /notificaciones?clabe={{origen}}

200 OK • 57 ms • 453 B

JSON

```

1 [
2   {
3     "id": 3,
4     "clabeCuenta": "002180790410379523",
5     "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
6     "fecha": "2026-09-27T06:15:46.863130Z"
7   },
8   {
9     "id": 1,
10    "clabeCuenta": "002180790410379523",
11    "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
12    "fecha": "2026-09-27T05:51:08.576373Z"
13  }
14 ]

```

Figura 15. Consulta de las notificaciones generadas para la cuenta origen.

Cuando algo sale mal

Se probaron diferentes solicitudes inválidas para comprobar el manejo de errores de la aplicación. Se obtuvieron respuestas 404 para una cuenta inexistente, 409 para una CLABE duplicada, 400 para solicitudes incompletas y 422 para transferencias no permitidas. También se comprobó que un SPEI sin Idempotency-Key fuera rechazado con 400. Finalmente, el saldo permaneció en 750, confirmando que las operaciones rechazadas no modificaron los datos.



Figura 16. Consulta de una CLABE inexistente y respuesta 404.



Figura 17. Intento de crear una cuenta con una CLABE repetida.



Figura 18. Solicitud para crear una cuenta sin proporcionar la CLABE.



Figura 19. Intento de transferencia con saldo insuficiente.



Figura 20. Intento de transferencia hacia la misma cuenta.



Figura 21. Intento de envío SPEI sin una clave de idempotencia.



Figura 22. Verificación del saldo después de las operaciones inválidas.

Persistencia de datos con volúmenes

Primero se detuvieron los servicios con `docker compose down` y se levantaron nuevamente. Al consultar la cuenta, el saldo de 750 seguía disponible, ya que el volumen de PostgreSQL se había conservado.

Después se utilizó `docker compose down -v`, eliminando también el volumen. Al volver a levantar el sistema, la misma cuenta respondió con 404 Not Found, comprobando que la información anterior había sido eliminada junto con el volumen.

```

fernandoreyesvazquez@MacBookAir mexi-banco % docker compose down
[+] down 3/3
✓ Container mexi-banco-app-1 Removed
✓ Container mexi-banco-db-1 Removed
✓ Network mexibanco Removed
fernandoreyesvazquez@MacBookAir mexi-banco % docker compose ps
NAME          IMAGE          COMMAND          SERVICE  CREATED      STATUS      PORTS
fernandoreyesvazquez@MacBookAir mexi-banco % docker compose up -d
[+] up 3/3
✓ Network mexibanco Created
✓ Container mexi-banco-db-1 Healthy
✓ Container mexi-banco-app-1 Started
fernandoreyesvazquez@MacBookAir mexi-banco % docker compose ps
NAME          IMAGE          COMMAND          SERVICE  CREATED      STATUS      PORTS
mexi-banco-app-1  mexi-banco-app  "java -jar app.jar"  app      20 seconds ago  Up 13 seconds (healthy)  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1  postgres:17-alpine  "docker-entrypoint.s..."  db      20 seconds ago  Up 19 seconds (healthy)  5432/tcp
  
```

Figura 23. Reinicio de los servicios de Mexi Banco conservando el volumen de datos.

Se detuvieron los servicios con `docker compose down` y posteriormente se levantaron nuevamente con `docker compose up -d`. Los contenedores fueron creados otra vez y tanto la aplicación como la base de datos alcanzaron el estado `healthy`.



Figura 24. Consulta de la cuenta después de reiniciar los servicios conservando el volumen.

Después de reiniciar los servicios se consultó nuevamente la cuenta origen. La aplicación respondió con 200 OK y el saldo continuó siendo 750, comprobando que los datos permanecieron almacenados aunque los contenedores fueron eliminados y creados nuevamente.

```
[fernandoreyesvazquez@MacBookAir mexi-banco % docker compose down -v
[+] down 4/4
✓ Container mexi-banco-app-1      Removed
✓ Container mexi-banco-db-1      Removed
✓ Network mexibanco              Removed
✓ Volume mexi-banco_mexibanco_datos Removed
[fernandoreyesvazquez@MacBookAir mexi-banco % docker compose up -d
[+] up 4/4
✓ Network mexibanco              Created
✓ Volume mexi-banco_mexibanco_datos Created
✓ Container mexi-banco-db-1      Healthy
✓ Container mexi-banco-app-1     Started

[fernandoreyesvazquez@MacBookAir mexi-banco % docker compose ps
NAME                IMAGE              COMMAND                  SERVICE  CREATED      STATUS      PORTS
mexi-banco-app-1    mexi-banco-app    "java -jar app.jar"      app      26 seconds ago Up 19 seconds (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine "docker-entrypoint.s..." db       26 seconds ago Up 25 seconds (healthy) 5432/tcp
```

Figura 25. Eliminación y nueva creación del volumen de datos de Mexi Banco.

Se detuvieron los servicios utilizando `docker compose down -v`. En este caso, además de los contenedores y la red, también se eliminó el volumen `mexi-banco_mexibanco_datos`. Al volver a levantar el sistema, Docker creó un nuevo volumen para la base de datos.



Figura 26. Consulta de la cuenta después de eliminar el volumen de la base de datos.

Después de eliminar el volumen y levantar nuevamente los servicios, se intentó consultar la misma cuenta utilizada anteriormente. La aplicación respondió con 404 Not Found, indicando que la cuenta ya no existía porque los datos almacenados anteriormente fueron eliminados junto con el volumen.

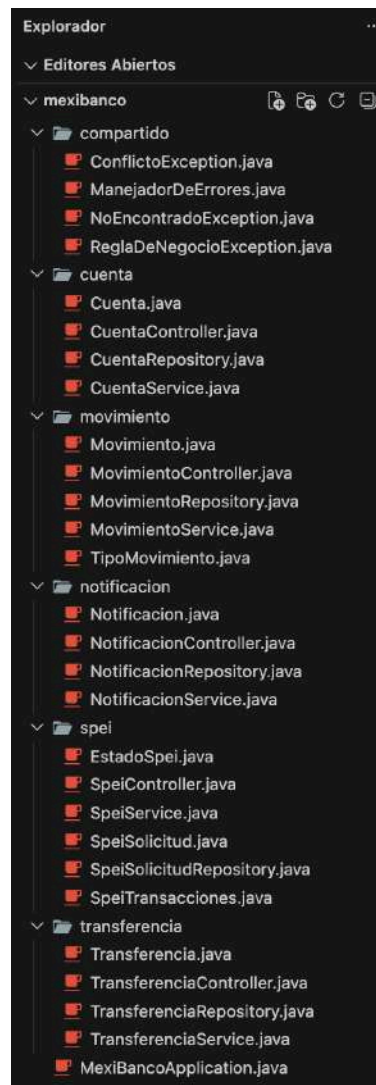


Figura 27. Organización de los módulos y capas del monolito Mexi Banco.

Se revisó el código ubicado en `src/main/java/mx/mexibanco`. El proyecto está dividido en los módulos `cuenta`, `movimiento`, `notificacion`, `spei` y `transferencia`, además de una carpeta `compartido` que contiene clases utilizadas para el manejo de errores.

Al revisar los módulos se observó una separación por capas. Por ejemplo, en el módulo `cuenta`, `CuentaController` recibe las peticiones HTTP, `CuentaService` contiene las reglas y operaciones de las cuentas, `CuentaRepository` permite acceder a la información almacenada en la base de datos y `Cuenta` representa la entidad con datos como la CLABE, titular y saldo.

También se observó que los módulos se relacionan entre sí. En una transferencia interna, `TransferenciaService` utiliza el servicio de cuentas para realizar el cargo y el abono, registra los movimientos y genera las notificaciones correspondientes. Estas operaciones se realizan dentro de una misma transacción.

En el módulo SPEI se encontró el manejo de la idempotencia. El sistema registra la `Idempotency-Key` antes de realizar el cobro y, si se vuelve a recibir la misma clave, recupera la operación que ya existe en lugar de cobrar nuevamente. Esto corresponde con lo observado anteriormente en las pruebas de Postman.

Finalmente, en la carpeta `compartido` se encuentra `ManejadorDeErrores`, que transforma diferentes excepciones de negocio en respuestas HTTP como 404, 409 y 422. Esto también coincide con los errores probados en la sección anterior de Postman.

```

cuenta > CuentaController.java
1  package mx.mexibanco.cuenta;
2
3  import jakarta.validation.Valid;
4  import jakarta.validation.constraints.NotBlank;
5  import jakarta.validation.constraints.NotNull;
6  import jakarta.validation.constraints.PositiveOrZero;
7  import org.springframework.http.HttpStatus;
8  import org.springframework.web.bind.annotation.*;
9
10 import java.math.BigDecimal;
11
12 /* Capa web: traduce HTTP a llamadas al servicio y nada mas. Sin reglas de negocio aqui. */
13 @RestController
14 @RequestMapping("/cuentas")
15 public class CuentaController {
16
17     private final CuentaService cuentas;
18
19     public CuentaController(CuentaService cuentas) {
20         this.cuentas = cuentas;
21     }
22
23     public record AltaCuenta(@NotBlank String clabe, @NotBlank String titular, @NotNull @PositiveOrZero BigDecimal saldoInicial)
24     {}
25
26     @PostMapping
27     @ResponseStatus(HttpStatus.CREATED)
28     public Cuenta abrir(@Valid @RequestBody AltaCuenta datos) {
29         return cuentas.abrir(datos.clabe(), datos.titular(), datos.saldoInicial());
30     }
31
32     @GetMapping("/{clabe}")
33     public Cuenta consultar(@PathVariable String clabe) {
34         return cuentas.consultar(clabe);
35     }
36 }
37

```

Figura 28. Controlador del módulo de cuentas de Mexi Banco.

Al revisar CuentaController.java se observaron los endpoints utilizados anteriormente en Postman para crear y consultar cuentas. El controlador recibe las solicitudes HTTP y delega la operación a CuentaService, sin contener directamente las reglas de negocio.

Compose y el Dockerfile

```
1 # Con un solo comando: docker compose up --build
2 # Lo que van a ver correr (S05, "de compose a orquestacion"):
3 # - 'app' resuelve 'db' por nombre (DNS embebido de Compose), sin ninguna IP en ningun lado.
4 # - 'app' espera a que 'db' este listo de verdad (depends_on + healthcheck), no solo que exista.
5 # - Las variables de entorno son la configuracion (factor III, ya visto en S04).
6 # - 'db' persiste en un volumen nombrado: si tiran el contenedor y lo vuelven a subir, los
7 #   datos siguen ahi; si tiran el volumen, no.
8 services:
9   db:
10     image: postgres:17-alpine
11     environment:
12       POSTGRES_DB: mexibanco
13       POSTGRES_USER: mexibanco
14       POSTGRES_PASSWORD: mexibanco
15     volumes:
16       - mexibanco_datos:/var/lib/postgresql/data
17     healthcheck:
18       test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
19       interval: 5s
20       timeout: 3s
21       retries: 5
22
23   app:
24     build: .
25     ports:
26       - "8080:8080"
27     environment:
28       DB_HOST: db
29       DB_PORT: "5432"
30       DB_NAME: mexibanco
31       DB_USER: mexibanco
32       DB_PASSWORD: mexibanco
33     depends_on:
34       db:
35         condition: service_healthy
36     healthcheck:
37       test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1"]
38       interval: 10s
39       timeout: 3s
40       retries: 5
41
42 networks:
43   default:
44     name: mexibanco
45
46 volumes:
47   mexibanco_datos:
```

Figura 29. Configuración de los servicios en el archivo docker-compose.yml.

En docker-compose.yml se identificaron los servicios app y db. PostgreSQL utiliza el volumen mexibanco_datos para la persistencia y cuenta con un healthcheck. La aplicación utiliza el puerto 8080, se conecta a la base mediante el nombre db y espera a que esta se encuentre en estado healthy.

```
1 # --- ETAPA 1: CONSTRUCCION ---
2 FROM eclipse-temurin:21-jdk-alpine AS builder
3 WORKDIR /app
4 COPY pom.xml .
5 COPY mvnw .
6 COPY .mvn .mvn
7 RUN ./mvnw dependency:go-offline -B
8 COPY src ./src
9 RUN ./mvnw clean package -DskipTests
10
11 # --- ETAPA 2: EJECUCION ---
12 FROM eclipse-temurin:21-jre-alpine
13 WORKDIR /app
14 COPY --from=builder /app/target/*.jar app.jar
15 EXPOSE 8080
16
17 ENTRYPOINT ["java", "-jar", "app.jar"]
18
```

Figura 30. Etapas de construcción y ejecución definidas en el Dockerfile.

El Dockerfile utiliza dos etapas. La primera emplea Java 21 JDK y Maven para compilar el proyecto y generar el archivo .jar. La segunda utiliza Java 21 JRE para ejecutar únicamente la aplicación compilada mediante java -jar app.jar.

Con estos archivos se observó cómo se construyen, conectan y ejecutan los componentes del sistema dentro de Docker.